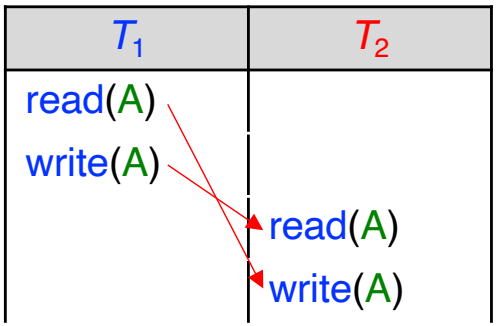
COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 22 EXERCISES
TRANSACTION MANAGEMENT: TRANSACTIONS

EXERCISE 1

Indicate which of the following schedules involving T_1 and T_2 is serial, serializable or not serializable.

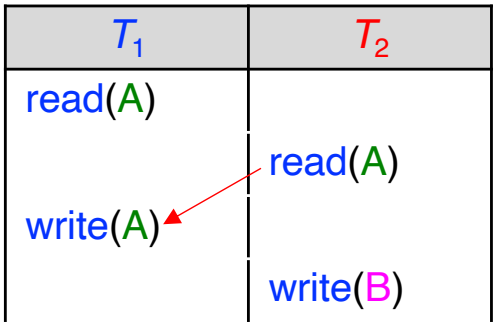
a)

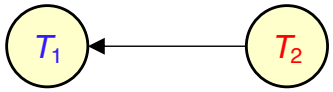


Precedence Graph

Serial

b)





Precedence Graph

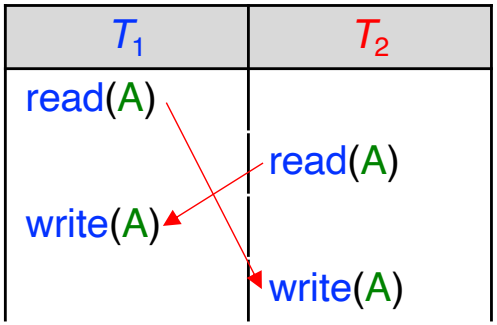
What is the equivalent serial schedule?

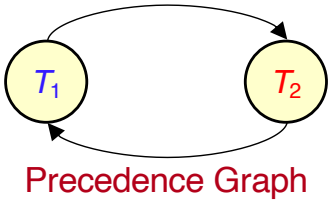
$T_2 T_1$

Serializable

EXERCISE 1 (CONT'D)

c)

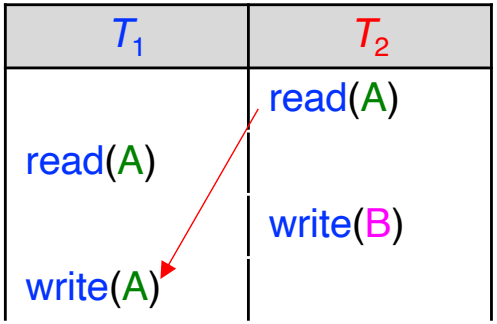


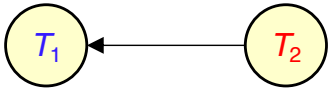


Precedence Graph

Not Serializable

d)





Precedence Graph

What is the equivalent serial schedule?

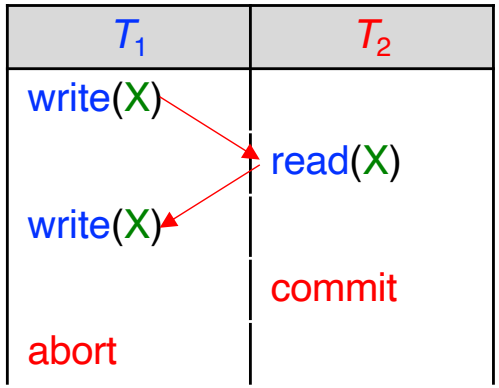
$T_2 T_1$

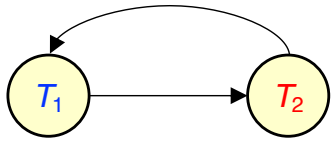
Serializable

EXERCISE 2

For each of the following schedules, state whether it is serializable, recoverable and cascadeless. Justify your answers.

a)





Precedence Graph

Serializable? No

Justification: The precedence graph has a cycle.

Recoverable? No

Justification: T_2 reads data item X written by T_1 and commits, but then T_1 aborts.

Cascadeless? No

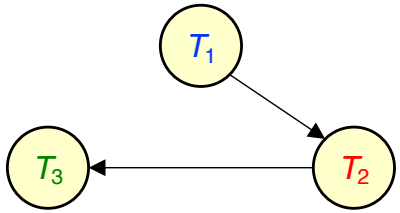
Justification: If T_1 aborts, T_2 must also abort.

Recoverable: If T_j reads a data item written by T_i and T_j commits after T_i.
Cascadeless: If T_j reads a data item written by T_i and T_i commits before read of T_j.

EXERCISE 2 (CONTD)

b)

T_1	T_2	T_3
	read(X)	
		write(X)
		commit
write(Y)		
commit		
	read(Y)	
	write(Z)	
	commit	



Precedence Graph

Serializable? Yes

Justification: There is no cycle in the precedence graph.

The equivalent serial schedule is $T_1 T_2 T_3$.

Recoverable? Yes

Justification: T_2 reads a data item written by T_1 and commits after T_1.

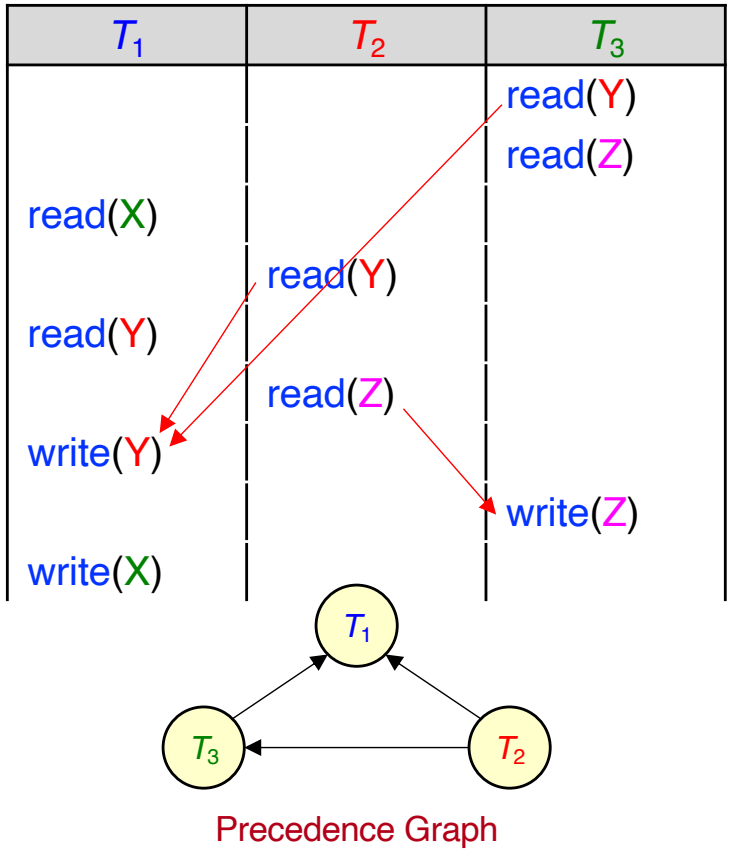
Cascadeless? Yes

Justification: Although T_2 reads a data item written by T_1, the commit of T_1 appears before the commit of T_2.

Recoverable: If T_j reads a data item written by T_i and T_j commits after T_i.
Cascadeless: If T_j reads a data item written by T_i and T_i commits before read of T_j.

EXERCISE 2 (CONT'D)

c)



Precedence Graph

Serializable? Yes

Justification: The precedence graph has no cycle.

The equivalent serial schedule is: $T_2 T_3 T_1$.

Recoverable? Yes

Justification: No transaction reads a data item *previously* written by another transaction.

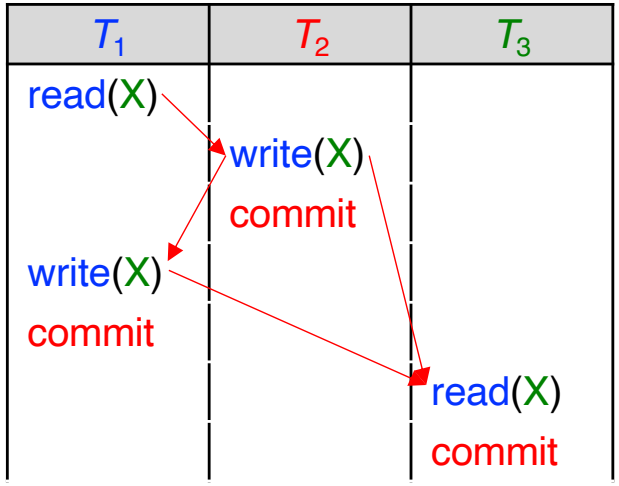
Cascadeless? Yes

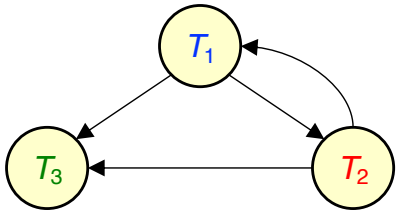
Justification: No transaction reads a data item *previously* written by another transaction.

Recoverable: If T_j reads a data item written by T_i and T_j commits after T_i.
Cascadeless: If T_j reads a data item written by T_i and T_i commits before read of T_j.

EXERCISE 2 (CONTD)

d)





Precedence Graph

Serializable? No

Justification: The precedence graph has a cycle.

Recoverable? Yes

Justification: Although T_3 reads a data item written by T_1, T_3 commits after T_1.

Cascadeless? Yes

Justification: The commit of T_1 appears before the commit of T_3.

Why recoverable and cascadeless?

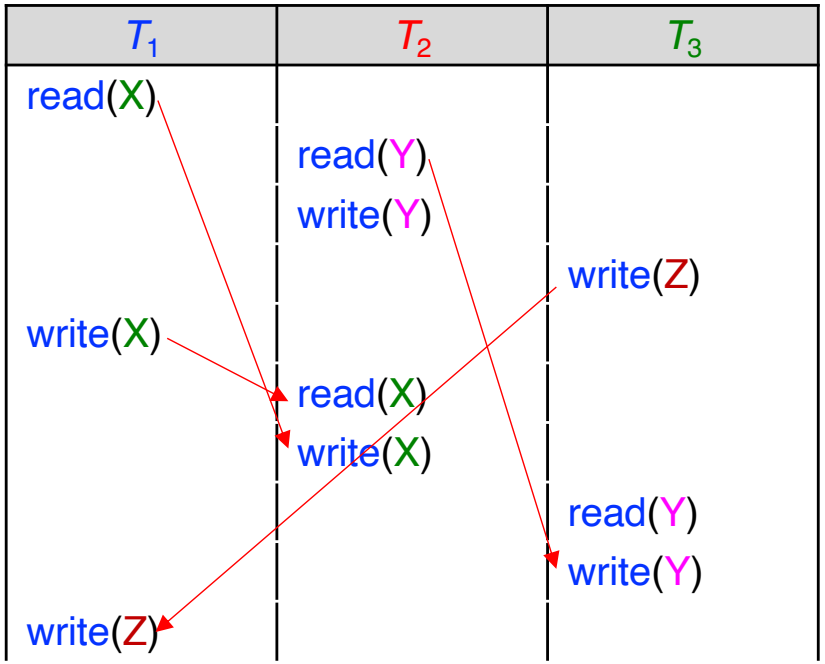
The write of T_2 can be ignored (called a blind write). It is not preceded by a read, and it is immediately overwritten by T_1. So, its value is never seen by any transaction.

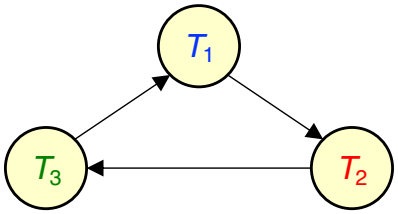
Recoverable: If T_j reads a data item written by T_i and T_j commits after T_i.
Cascadeless: If T_j reads a data item written by T_i and T_i commits before read of T_j.

EXERCISE 3

For each of the following schedules, answer the questions.

a) Is the schedule serializable?





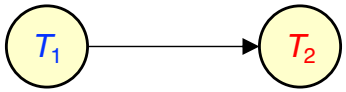
The schedule is not serializable since there is a cycle in the precedence graph.

There is no equivalent serial schedule because of the cycle.

EXERCISE 3 (CONTD)

b) Is the schedule serializable, recoverable and cascadeless?

T_1	T_2
	write(B)
read(A)	
write(A)	
	read(A)
commit	
	commit



Serializable? Yes

There is no cycle in the precedence graph.

The equivalent serial schedule is $T_1\ T_2$.

Recoverable? Yes

T_2 reads data item A written by T_1 and T_2 commits after T_1.

Cascadeless? No

T_2 reads data item A written by T_1 before the commit of T_1.

How would you place the commit statements to make the schedule cascadeless?

T_1	T_2
	write(B)
read(A)	
write(A)	
commit	
	read(A)
	commit

Recoverable: If T_j reads a data item written by T_i and T_j commits after T_i.
Cascadeless: If T_j reads a data item written by T_i and T_i commits before read of T_j.

EXERCISE 3 (CONTD)

c) Is the schedule recoverable and cascadeless?

T_1	T_2
	read(A)
read(A)	
write(A)	
	write(B)
	commit
commit	

Recoverable? Yes

No transaction reads data items after they have been written by the other transaction.

Cascadeless? Yes

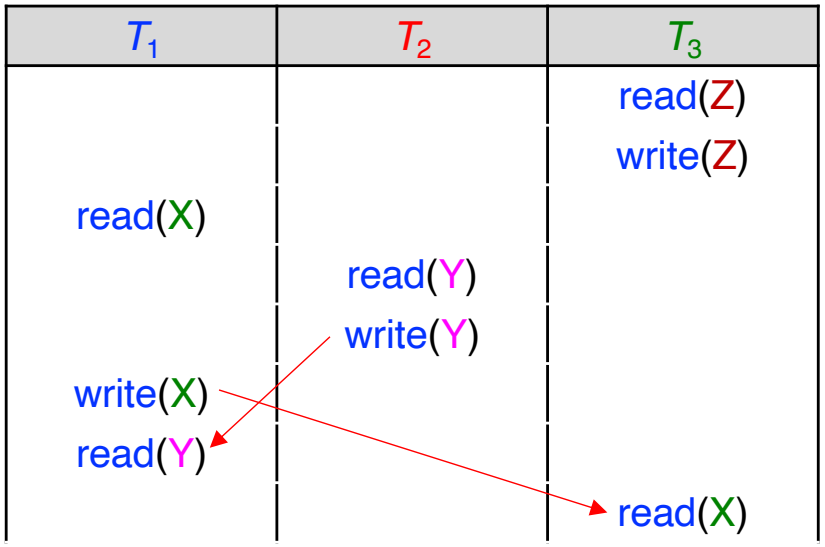
No transaction reads data items after they have been written by the other transaction.

Recoverable: If T_j reads a data item written by T_i and T_j commits after T_i.
Cascadeless: If T_j reads a data item written by T_i and T_i commits before read of T_j.

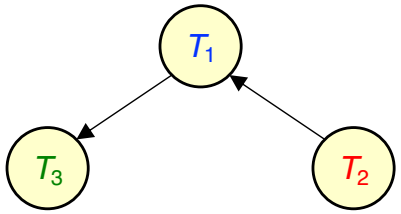
EXERCISE 4

Consider the following schedule consisting of transactions T_1, T_2 and T_3.

a) Show that the schedule is serializable by constructing the precedence graph.



Precedence Graph



b) What is the equivalent serial schedule?

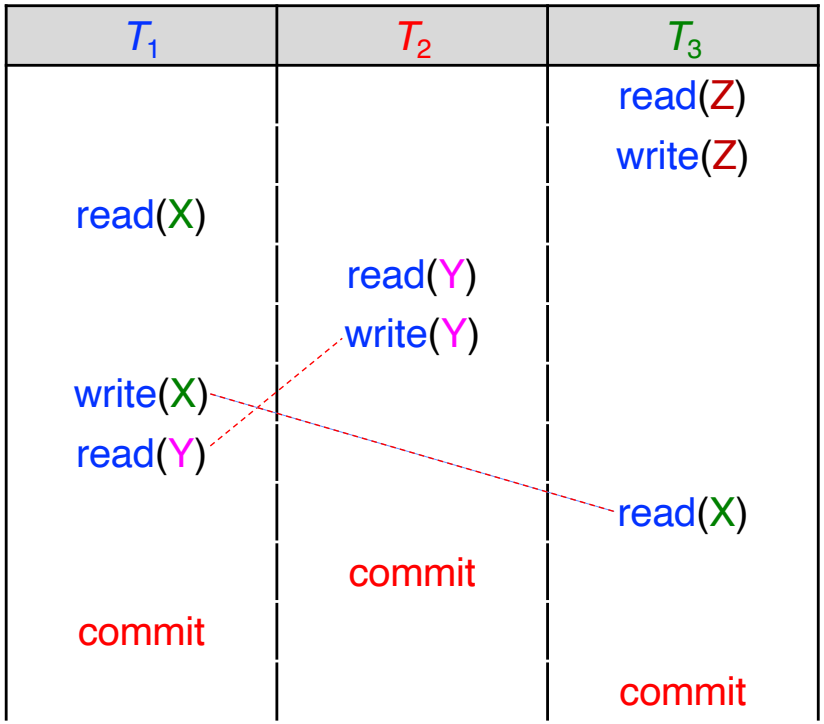
$T_2 T_1 T_3$

EXERCISE 4 (CONT'D)

c) Modify the original schedule so it becomes recoverable, but not cascadeless by adding commit instructions *in the correct order* to the end the schedule (i.e., after the instructions of all transactions have executed).

Recoverable: If T_j reads a data item written by T_i and commits after T_i.

T_1 reads Y written by T_2
$\Rightarrow$ T_1 must commit after T_2.
T_2 does not read any data items written by other transactions.
T_3 reads X written by T_1
$\Rightarrow$ T_3 must commit after T_1.



EXERCISE 4 (CONT'D)

d) Modify the original schedule so it becomes both recoverable and cascadeless by adding commit instructions in the appropriate locations in the schedule.

Cascadeless: If T_j reads a data item written by T_i and T_i commits before the read of T_j.

- The commit of T_1 must appear *before* the read(X) of T_3.
- The commit of T_2 must appear *before* the read(Y) of T_1.
- The commit of T_3 must appear at the end of T_3.

T_1	T_2	T_3
		read(Z)
		write(Z)
read(X)		
	read(Y)	
	write(Y)	
	commit	
write(X)		
read(Y)		
commit		
		read(X)
		commit